

Value Source Annotation Overview



- **JUnit** offers `@ValueSource` to **supply parameters** to a **test method**.
- Works together with `@ParameterizedTest` annotation.
- Allows a **test** to **run multiple times** with **different single-value inputs**.
- Useful for simple, **one-parameter test scenarios**.

 **by Rodrigo Martins Pagliares**

Last update: 11/27/2025

Example implemented in this lesson



Print Values

Print each received value using `System.out.println` (for demonstration).



Validate Input

Use `assertNotNull(phoneNumber)` to ensure input is not null.



Independent Testing

Each execution validates one piece of data independently.



Identify Issues

Helps identify problematic values through repeated structured runs.



Creating a Parameterized Test with @ValueSource

- **Define** a **test method** annotated with `@ParameterizedTest`.
- Add `@ValueSource` to **supply values** in an **array-like format** (Array of Strings in this example).
- Method **must accept exactly one parameter**.
- **Example:** a String `phoneNumber` **parameter**.

```
133     @ParameterizedTest
134     @ValueSource(strings = {"+55 11 98765-4321", "+55 21 99612-7845", "+55 35 99148-2307"})
135     public void demonstrateValueSourceAnnotation(String phoneNumber) {
136         System.out.println("Telephone number: " + phoneNumber);
137         assertNotNull(phoneNumber);
138     }
```

Providing Input Values

```
@ValueSource(strings = {"+55 11 98765-4321", "+55 21 99612-7845",  
                        "+55 35 99148-2307"})  
public void demonstrateValueSourceAnnotation(String phoneNumber) {...}
```

- Each **value** appears **inside quotes** and is **separated by commas**.
- **Test runs once per** provided **value**.
- Enables **quick testing** of **multiple inputs** without writing separate methods.

```
133      @ParameterizedTest  
134      @ValueSource(strings = {"+55 11 98765-4321", "+55 21 99612-7845", "+55 35 99148-2307"})  
135  ▶     public void demonstrateValueSourceAnnotation(String phoneNumber) {  
136         System.out.println("Telephone number: " + phoneNumber);  
137         assertNotNull(phoneNumber);  
138     }
```

Running the test method annotated with @ValueSource

The screenshot shows an IDE window with the following components:

- Editor:** Displays the `CalculatorTest.java` file. The code is as follows:

```
12 public class CalculatorTest {  
133     @ParameterizedTest  
134     @ValueSource(strings = {"+55 11 98765-4321", "+55 21 99612-7845", "+55 35 99148-2307"})  
135     public void demonstrateTheUseOfValueSourceAnnotation(String phoneNumber) {  
136         System.out.println("Telephone number: " + phoneNumber);  
137         assertNotNull(phoneNumber);  
138     }  
139 }
```
- Run Console:** Shows the execution of the test method `CalculatorTest.demonstrateValueSourceAnnotation`. The output is:

```
/Library/Java/JavaVirtualMachines/temurin-25.jdk/Contents/Home  
Telephone number: +55 11 98765-4321  
Telephone number: +55 21 99612-7845  
Telephone number: +55 35 99148-2307  
  
Process finished with exit code 0
```
- Test Results:** A table at the bottom left summarizes the test execution:

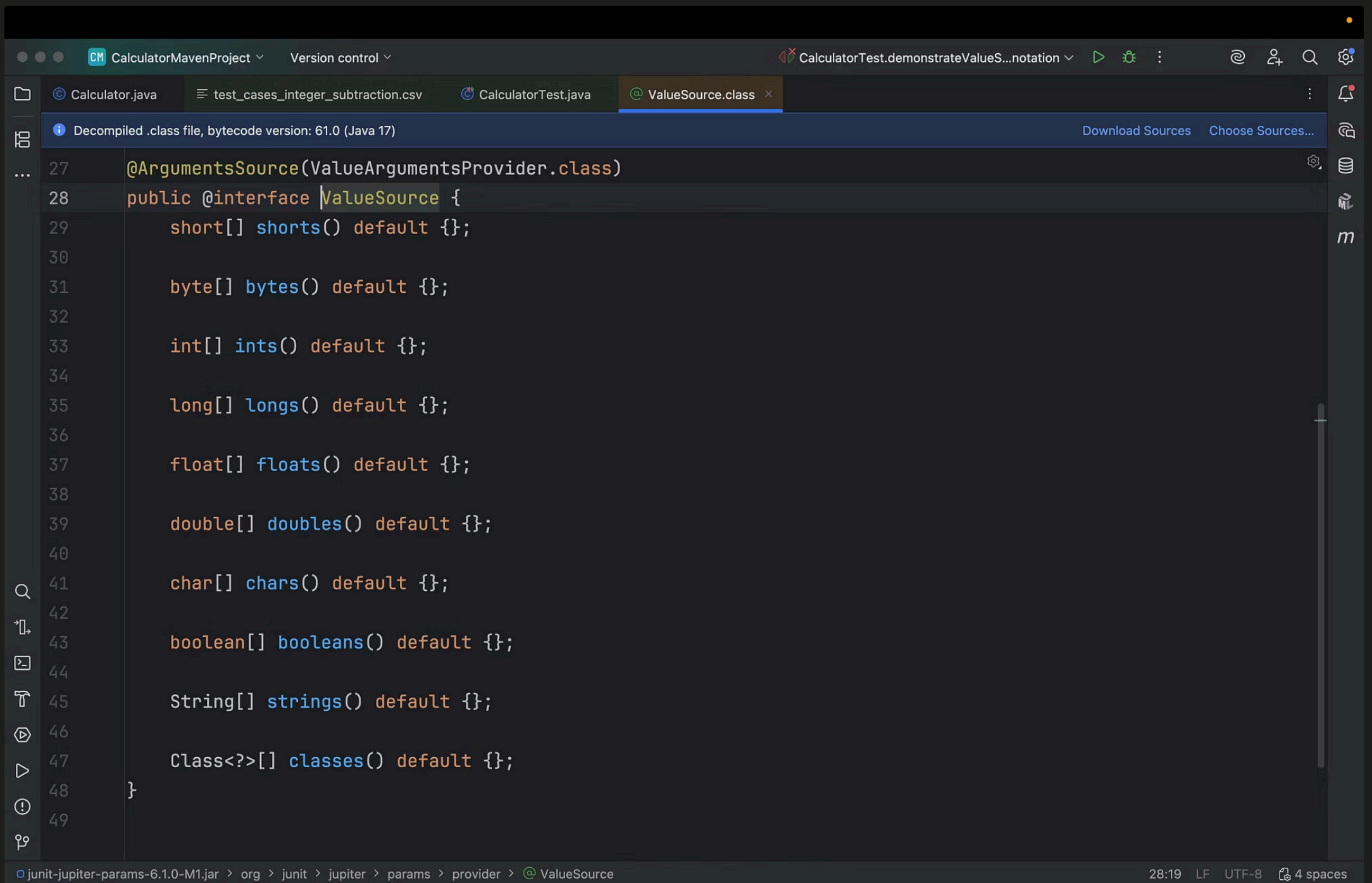
Test Name	Duration
Test operations in Calculator class (org.example)	11 ms
demonstrateValueSourceAnnotation(String)	11 ms
[1] "+55 11 98765-4321"	11 ms
[2] "+55 21 99612-7845"	
[3] "+55 35 99148-2307"	

3 tests executed with **3** different **cell-phone numbers**.

All **3** tests **passed**!

Inside the code of the @ValueSource annotation

By inspecting the source code of the `ValueSource` class we can see that, in addition to a **array** of **Strings**, we can also use other types as well.



The screenshot shows an IDE window with the following tabs: `CalculatorMavenProject`, `Version control`, `CalculatorTest.demonstrateValueS...notation`, `Calculator.java`, `test_cases_integer_subtraction.csv`, `CalculatorTest.java`, and `@ ValueSource.class`. The `@ ValueSource.class` tab is active, displaying the decompiled source code of the `@ValueSource` annotation. The code is as follows:

```
27 @ArgumentsSource(ValueArgumentsProvider.class)
28 public @interface ValueSource {
29     short[] shorts() default {};
30
31     byte[] bytes() default {};
32
33     int[] ints() default {};
34
35     long[] longs() default {};
36
37     float[] floats() default {};
38
39     double[] doubles() default {};
40
41     char[] chars() default {};
42
43     boolean[] booleans() default {};
44
45     String[] strings() default {};
46
47     Class<?>[] classes() default {};
48 }
49
```

The IDE interface includes a sidebar on the left with icons for file explorer, search, and other tools. The bottom status bar shows the file path: `junit-jupiter-params-6.1.0-M1.jar > org > junit > jupiter > params > provider > @ ValueSource`, along with the line number 28:19, LF line ending, UTF-8 encoding, and 4 spaces indentation.



Supported Data Types

@ValueSource works with several **primitive** and **object** types:

Strings, ints, longs, doubles, shorts, bytes, chars, booleans.

- Flexible for many **scenarios requiring** simple, **single-value inputs**.
- Ideal for small data sets and quick validation.

Conclusion



- `@ValueSource` provides a clean way to test methods repeatedly with different inputs.
- Best for simple **one-parameter tests** rather than complex scenarios.
- Helps reduce boilerplate and makes test suites more expressive and **concise**.
- Easy starting point for **parameterized testing** in **JUnit**.

Open Floor Discussion



**@ValueSource vs
@CsvSource**

When should you choose
@ValueSource over
@CsvSource?

Early Development

How might single-value
testing simplify early
development stages?

Edge Cases

Could @ValueSource help
expose edge cases you
may overlook?